T3000 API & Protocol

Control & Automation API · built Nov 13, 2026

Control & Automation API

Overview

The Control & Automation API manages programs, schedules, PIDs, holidays, and annual routines for automated building control. Use these endpoints to configure time-based control, logic programs, and temperature regulation.

Features

- **Programs**: Custom control logic and sequences
- **Schedules**: Time-based control patterns
- **PIDs**: Temperature and pressure control loops
- **Holidays**: Special day schedules
- **Annual Routines**: Recurring yearly events

Common Operations

Get All Schedules

TYPESCRIPT

```typescript
const schedules = await fetch('http://localhost:9103/api/t3_device/devices/1/schedules')
  .then(res => res.json());

console.log(schedules);
// [
//   {
//     "id": 1,
//     "device_id": 1,
//     "panel": 5,
//     "index": 1,
//     "label": "Office Hours",
//     "auto_manual": 0,
//     "output": 1,
//     "holiday_id": 0,
//     "status": "ON",
//     "override": 0
//   }
// ]
```

Get Schedule Details

TYPESCRIPT

```typescript
const schedule = await fetch('http://localhost:9103/api/t3_device/schedules/1')
  .then(res => res.json());
```

Update Schedule

TYPESCRIPT

```typescript
await fetch('http://localhost:9103/api/t3_device/schedules/1', {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    label: "Office Hours - Updated",
    auto_manual: 1,
    output: 100
  })
});
```

Get All Programs

TYPESCRIPT

```typescript
const programs = await fetch('http://localhost:9103/api/t3_device/devices/1/programs')
  .then(res => res.json());
```

Update Program Code

TYPESCRIPT

```typescript
await fetch('http://localhost:9103/api/t3_device/programs/1', {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    label: "VAV Control",
    description: "Variable Air Volume Logic",
    code: "10 IF IN1 > VAR1 THEN OUT1 = 100\n20 ELSE OUT1 = 0\n30 END"
  })
});
```

Get PIDs

TYPESCRIPT

```typescript
const pids = await fetch('http://localhost:9103/api/t3_device/devices/1/pids')
  .then(res => res.json());
```

Schedule Endpoints

GET /api/t3_device/devices/:id/schedules

Description: Get all schedules for a device

Parameters:

- `id` (path, integer) - Device ID

Request:

```http
GET /api/t3_device/devices/1/schedules HTTP/1.1
```

Response:

```json
[
  {
    "id": 1,
    "device_id": 1,
    "panel": 5,
    "index": 1,
    "label": "Office Hours",
    "auto_manual": 0,
    "output": 1,
    "holiday_id": 0,
    "status": "ON",
    "override": 0,
    "times": {
      "monday": [
        { "start": "08:00", "end": "18:00", "value": 1 }
      ],
      "tuesday": [
        { "start": "08:00", "end": "18:00", "value": 1 }
      ]
      // ... other days
    }
  }
]
```

Used By:

- ScheduleTable.tsx
- ScheduleCalendar.tsx

Handler: `api/src/t3_device/routes.rs::get_schedules_by_device`

GET /api/t3_device/schedules/:id

Description: Get single schedule by ID

Parameters:

- `id` (path, integer) - Schedule ID

Request:

```http
GET /api/t3_device/schedules/1 HTTP/1.1
```

Response:

```json
{
  "id": 1,
  "device_id": 1,
  "panel": 5,
  "index": 1,
  "label": "Office Hours",
  "auto_manual": 0,
  "output": 1,
  "holiday_id": 0,
  "status": "ON",
  "override": 0,
  "times": {
    // ... weekly schedule
  }
}
```

Used By:

- ScheduleDetail.tsx

Errors:

- `404` - Schedule not found

PUT /api/t3_device/schedules/:id

Description: Update schedule configuration

Parameters:

- `id` (path, integer) - Schedule ID

Request:

HTTP

```
PUT /api/t3_device/schedules/1 HTTP/1.1
Content-Type: application/json

{
  "label": "Office Hours - Winter",
  "auto_manual": 0,
  "output": 1,
  "times": {
    "monday": [
      { "start": "07:00", "end": "19:00", "value": 1 }
    ]
  }
}
```

Updatable Fields:

- `label`, `auto_manual`, `output`, `holiday_id`, `status`, `override`, `times`

Response:

JSON

```
{
  "success": true,
  "id": 1,
  "updated_at": "2026-01-09T10:30:00Z"
}
```

Used By:

- EditScheduleDialog.tsx

POST /api/t3_device/schedules

Description: Create new schedule

Request:

```HTTP
POST /api/t3_device/schedules HTTP/1.1
Content-Type: application/json

{
  "device_id": 1,
  "panel": 5,
  "label": "Night Setback",
  "auto_manual": 0,
  "output": 0
}
```

Required Fields:

- `device_id`, `panel`, `label`

Response:

```JSON
{
  "id": 13,
  "device_id": 1,
  "label": "Night Setback",
  "created_at": "2026-01-09T10:35:00Z"
}
```

Used By:

- AddScheduleDialog.tsx

DELETE /api/t3_device/schedules/:id

Description: Delete schedule

Parameters:

- `id` (path, integer) - Schedule ID

Request:

```
HTTP
DELETE /api/t3_device/schedules/1 HTTP/1.1
```

Response:

```
HTTP
HTTP/1.1 204 No Content
```

Used By:

- DeleteScheduleDialog.tsx

Program Endpoints

GET /api/t3_device/devices/:id/programs

Description: Get all programs for a device

Parameters:

- `id` (path, integer) - Device ID

Request:

```
HTTP
GET /api/t3_device/devices/1/programs HTTP/1.1
```

Response:

JSON


```
[
  {
    "id": 1,
    "device_id": 1,
    "panel": 5,
    "index": 1,
    "label": "VAV Control",
    "description": "Variable Air Volume Logic",
    "auto_manual": 0,
    "status": "RUNNING",
    "code": "10 IF IN1 > VAR1 THEN OUT1 = 100\n20 ELSE OUT1 = 0\n30 END",
    "size": 52,
    "execution_time": 15
  }
]
```


Used By:

- ProgramTable.tsx

GET /api/t3_device/programs/:id

Description: Get single program by ID

Parameters:

- `id` (path, integer) - Program ID

Request:

HTTP

```
GET /api/t3_device/programs/1 HTTP/1.1
```

Response:

JSON

```
{
  "id": 1,
  "device_id": 1,
  "panel": 5,
  "index": 1,
  "label": "VAV Control",
  "description": "Variable Air Volume Logic",
  "code": "10 IF IN1 > VAR1 THEN OUT1 = 100\n20 ELSE OUT1 = 0\n30 END",
  "status": "RUNNING"
}
```

Used By:

- ProgramEditor.tsx

PUT /api/t3_device/programs/:id

Description: Update program code and settings

Parameters:

- `id` (path, integer) - Program ID

Request:

HTTP

```
PUT /api/t3_device/programs/1 HTTP/1.1
Content-Type: application/json

{
  "label": "VAV Control v2",
  "description": "Updated logic for winter mode",
  "code": "10 IF IN1 > VAR1 THEN OUT1 = 100\n15 IF IN2 < 40 THEN OUT1 = 50\n20 ELSE OUT1
  "auto_manual": 0
}
```

Updatable Fields:

- `label`, `description`, `code`, `auto_manual`, `status`

Response:

JSON

```
{
  "success": true,
  "id": 1,
  "compiled": true,
  "errors": []
}
```

Used By:

- ProgramEditor.tsx

Errors:

- `400` - Syntax error in program code
- `404` - Program not found

POST /api/t3_device/programs

Description: Create new program

Request:

HTTP

```
POST /api/t3_device/programs HTTP/1.1
Content-Type: application/json

{
  "device_id": 1,
  "panel": 5,
  "label": "New Logic",
  "description": "Custom control sequence",
  "code": ""
}
```

Response:

```JSON
{
  "id": 5,
  "device_id": 1,
  "label": "New Logic",
  "created_at": "2026-01-09T11:00:00Z"
}
```

Used By:

- AddProgramDialog.tsx

DELETE /api/t3_device/programs/:id

Description: Delete program

Parameters:

- `id` (path, integer) - Program ID

Request:

```HTTP
DELETE /api/t3_device/programs/1 HTTP/1.1
```

Response:

```HTTP
HTTP/1.1 204 No Content
```

PID Endpoints

GET /api/t3_device/devices/:id/pids

Description: Get all PID loops for a device

Parameters:

- `id` (path, integer) - Device ID

Request:

```HTTP
GET /api/t3_device/devices/1/pids HTTP/1.1
```

Response:

```JSON
[
  {
    "id": 1,
    "device_id": 1,
    "panel": 5,
    "index": 1,
    "label": "Zone Temperature",
    "input": 1,
    "setpoint": 72.0,
    "output": 1,
    "proportional": 10.0,
    "integral": 5.0,
    "derivative": 1.0,
    "bias": 0.0,
    "auto_manual": 0,
    "status": "RUNNING"
  }
]
```

Used By:

- PIDTable.tsx

GET /api/t3_device/pids/:id

Description: Get single PID by ID

Parameters:

- `id` (path, integer) - PID ID

Request:

```http
GET /api/t3_device/pids/1 HTTP/1.1
```

Response:

```json
{
  "id": 1,
  "label": "Zone Temperature",
  "input": 1,
  "setpoint": 72.0,
  "output": 1,
  "proportional": 10.0,
  "integral": 5.0,
  "derivative": 1.0
}
```

Used By:

- PIDTuning.tsx

PUT /api/t3_device/pids/:id

Description: Update PID parameters

Parameters:

- `id` (path, integer) - PID ID

Request:

```HTTP
PUT /api/t3_device/pids/1 HTTP/1.1
Content-Type: application/json

{
  "setpoint": 70.0,
  "proportional": 12.0,
  "integral": 6.0,
  "derivative": 1.5,
  "auto_manual": 0
}
```

Updatable Fields:

- `label`, `input`, `setpoint`, `output`, `proportional`, `integral`, `derivative`, `bias`, `auto_manual`

Response:

```JSON
{
  "success": true,
  "id": 1,
  "updated_at": "2026-01-09T11:15:00Z"
}
```

Used By:

- PIDTuning.tsx

Holiday Endpoints

GET /api/t3_device/devices/:id/holidays

Description: Get all holidays for a device

Parameters:

- `id` (path, integer) - Device ID

Request:

```HTTP
GET /api/t3_device/devices/1/holidays HTTP/1.1
```

Response:

```JSON
[
  {
    "id": 1,
    "device_id": 1,
    "panel": 5,
    "index": 1,
    "label": "New Year",
    "auto_manual": 0,
    "value": 0,
    "date": "2026-01-01"
  }
]
```

Used By:

- HolidayCalendar.tsx

PUT /api/t3_device/holidays/:id

Description: Update holiday

Parameters:

- `id` (path, integer) - Holiday ID

Request:

```HTTP
PUT /api/t3_device/holidays/1 HTTP/1.1
Content-Type: application/json

{
  "label": "New Year's Day",
  "date": "2027-01-01",
  "auto_manual": 0,
  "value": 0
}
```

Response:

```JSON
{
  "success": true,
  "id": 1
}
```

Used By:

- EditHolidayDialog.tsx

Annual Routine Endpoints

GET /api/t3_device/devices/:id/annual-routines

Description: Get all annual routines for a device

Parameters:

- `id` (path, integer) - Device ID

Request:

```HTTP
GET /api/t3_device/devices/1/annual-routines HTTP/1.1
```

Response:

JSON

```
[
  {
    "id": 1,
    "device_id": 1,
    "panel": 5,
    "index": 1,
    "label": "Daylight Savings Start",
    "auto_manual": 0,
    "value": 1,
    "month": 3,
    "day": 14
  }
]
```

Used By:

- AnnualRoutineTable.tsx

PUT /api/t3_device/annual-routines/:id

Description: Update annual routine

Parameters:

- `id` (path, integer) - Annual routine ID

Request:

HTTP

```
PUT /api/t3_device/annual-routines/1 HTTP/1.1
Content-Type: application/json

{
  "label": "DST Start",
  "month": 3,
  "day": 14,
  "value": 1
}
```

Response:

```JSON
{
  "success": true,
  "id": 1
}
```

Used By:

- EditAnnualRoutineDialog.tsx

Implementation Details

Database Schema

SQL

```sql
CREATE TABLE SCHEDULES (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id INTEGER NOT NULL,
    panel INTEGER NOT NULL,
    index_ INTEGER NOT NULL,
    label TEXT,
    auto_manual INTEGER DEFAULT 0,
    output INTEGER DEFAULT 0,
    holiday_id INTEGER DEFAULT 0,
    status TEXT,
    override INTEGER DEFAULT 0,
    FOREIGN KEY (device_id) REFERENCES DEVICES(id) ON DELETE CASCADE
);

CREATE TABLE PROGRAMS (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id INTEGER NOT NULL,
    panel INTEGER NOT NULL,
    index_ INTEGER NOT NULL,
    label TEXT,
    description TEXT,
    auto_manual INTEGER DEFAULT 0,
    status TEXT DEFAULT 'STOPPED',
    code TEXT,
    size INTEGER DEFAULT 0,
    execution_time INTEGER DEFAULT 0,
    FOREIGN KEY (device_id) REFERENCES DEVICES(id) ON DELETE CASCADE
);

CREATE TABLE PIDS (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id INTEGER NOT NULL,
    panel INTEGER NOT NULL,
    index_ INTEGER NOT NULL,
    label TEXT,
    input INTEGER,
    setpoint REAL DEFAULT 0,
    output INTEGER,
    proportional REAL DEFAULT 10.0,
    integral REAL DEFAULT 5.0,
```

```
    derivative REAL DEFAULT 1.0,
    bias REAL DEFAULT 0.0,
    auto_manual INTEGER DEFAULT 0,
    status TEXT,
    FOREIGN KEY (device_id) REFERENCES DEVICES(id) ON DELETE CASCADE
);

CREATE TABLE HOLIDAYS (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id INTEGER NOT NULL,
    panel INTEGER NOT NULL,
    index_ INTEGER NOT NULL,
    label TEXT,
    auto_manual INTEGER DEFAULT 0,
    value INTEGER DEFAULT 0,
    date TEXT,
    FOREIGN KEY (device_id) REFERENCES DEVICES(id) ON DELETE CASCADE
);

CREATE TABLE ANNUAL_ROUTINES (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id INTEGER NOT NULL,
    panel INTEGER NOT NULL,
    index_ INTEGER NOT NULL,
    label TEXT,
    auto_manual INTEGER DEFAULT 0,
    value INTEGER DEFAULT 0,
    month INTEGER,
    day INTEGER,
    FOREIGN KEY (device_id) REFERENCES DEVICES(id) ON DELETE CASCADE
);
```

Program Language Syntax

T3000 programs use a BASIC-like syntax:

```BASIC
10 IF IN1 > 72 THEN OUT1 = 100
20 ELSE OUT1 = 0
30 END
```

Commands: IF, THEN, ELSE, FOR, NEXT, GOTO, GOSUB, RETURN, END

Operators: =, <, >, <=, >=, <>, +, -, *, /

References: IN1-IN16, OUT1-OUT8, VAR1-VAR64

PID Tuning Guidelines

- **Proportional**: Initial response strength (typical: 5-20)
- **Integral**: Eliminates steady-state error (typical: 2-10)
- **Derivative**: Dampens oscillations (typical: 0.5-3)

TypeScript Types

TYPESCRIPT

```typescript
export interface Schedule {
  id: number;
  device_id: number;
  panel: number;
  index: number;
  label: string;
  auto_manual: number;
  output: number;
  holiday_id: number;
  status: string;
  override: number;
  times?: ScheduleTimes;
}

export interface Program {
  id: number;
  device_id: number;
  panel: number;
  index: number;
  label: string;
  description: string;
  code: string;
  status: string;
  auto_manual: number;
}

export interface PID {
  id: number;
  device_id: number;
  panel: number;
  index: number;
  label: string;
  input: number;
  setpoint: number;
  output: number;
  proportional: number;
  integral: number;
  derivative: number;
  bias: number;
  auto_manual: number;
```

```
  status: string;
}
```